

# CMMS-BioAI

## Plan Maestro de Integracion Modulo de Herramientas

Documento de planificacion para la integracion del modulo HERRAMIENTAS al sistema CMMS-BioAI, incluyendo correcciones de errores existentes, migraciones de base de datos y mejoras de la interfaz de usuario.

Fecha: 6 de junio de 2026

Repositorio: [github.com/khitisakamachisa-collab/cmmsbioai](https://github.com/khitisakamachisa-collab/cmmsbioai)

# 1. Diagnostico del Estado Actual

Antes de agregar nuevas funcionalidades, es fundamental entender los problemas existentes en el proyecto. Durante las sesiones anteriores se realizaron modificaciones parciales que dejaron inconsistencias entre modelos, schemas, rutas y frontend. A continuacion se detalla cada problema encontrado.

## 1.1 Error Critico: OtRepuestoUtilizado eliminado

El archivo `models/repuestos.py` fue reescrito en una sesion anterior para agregar campos nuevos al modelo Repuesto, pero se elimino accidentalmente la clase `OtRepuestoUtilizado`. Esta clase es una tabla intermedia que relaciona Ordenes de Trabajo con Repuestos (registra cuantos repuestos se usaron en cada OT). El archivo `api/routes/ordenes.py` importa esta clase en la linea 8, causando un `ImportError` que impide el inicio del servidor.

## 1.2 Error Critico: FK ordenestrabajo no existe

El modelo `DocumentoAdjunto` define una foreign key como `foreign_key="ordenestrabajo.id"`, pero la tabla real se llama "ordentrabajo" (sin la "e"). Esto se debe a que la clase `OrdenTrabajo` no define explicitamente `__tablename__`, por lo que `SQLModel` genera automaticamente el nombre en minusculas como "ordentrabajo". Este error causa un `NoReferencedTableError` en el inicio del servidor.

## 1.3 OtRepuestoUtilizado no exportado

En `models/__init__.py`, la importacion de `OtRepuestoUtilizado` esta comentada. Esto significa que incluso si la clase existiera en `repuestos.py`, `SQLModel` no la registraria en su metadata y la tabla `otrepuestoutilizado` nunca se crearia en la base de datos, rompiendo la funcionalidad de repuestos utilizados en las OTs.

## 1.4 Schemas de Repuesto desactualizados

Los schemas Pydantic en `schemas/repuesto.py` solo contienen 7 campos basicos, pero el modelo Repuesto ahora tiene 14 campos. Los campos faltantes en los schemas son: `numero_serie`, `especificaciones_tecnicas`, `categoria`, `estado_uso`, `proveedor_ultimo`, `fecha_ultima_entrada` y `precio_referencia`. Sin estos campos en los schemas, la API no puede crear ni leer registros completos.

## 1.5 Modulo Herramientas inexistente

El modulo de Herramientas no existe en absoluto en el repositorio actual. No hay modelo, schema, ruta ni componente frontend. Se necesitan crear todos los archivos desde cero, siguiendo el patron ya establecido por Repuestos.

## 1.6 Tablas sin \_\_tablename\_\_ explícito

Los modelos `Equipo` y `OrdenTrabajo` no definen `__tablename__`. `SQLModel` genera automaticamente el nombre en minusculas (`equipo`, `ordentrabajo`), lo cual es funcional pero fragil. Si alguien renombra la clase, el nombre de la tabla cambia y todas las foreign keys se rompen. La mejor practica es definir `__tablename__` explicitamente en todos los modelos.

### 1.7 Sistema de migraciones inexistente

El archivo `database.py` solo usa `SQLModel.metadata.create_all(engine)`, el cual crea tablas nuevas pero NO agrega columnas a tablas existentes. Cuando se agregan campos nuevos al modelo Repuesto, la base de datos existente no se actualiza. Se necesitan funciones de migracion manual con `ALTER TABLE` para cada columna nueva.

## 2. Resumen de Archivos a Modificar

A continuacion se listan todos los archivos que necesitan creacion o modificacion, organizados por fase de implementacion. Los archivos marcados con "NUEVO" no existen actualmente en el repositorio.

[illegible]

### 3. Plan de Implementacion por Fases

El plan se divide en 5 fases secuenciales. Cada fase debe completarse y verificarse antes de pasar a la siguiente. Al final de cada fase, el backend debe arrancar sin errores y las funcionalidades existentes deben seguir operando normalmente.

#### Fase 1: Corregir errores criticos del backend

Esta fase resuelve los dos errores que impiden que el servidor arranque. Sin esta correccion, ninguna otra modificacion es posible.

##### Paso 1.1: Restaurar OtRepuestoUtilizado en models/repuestos.py

- Agregar `__tablename__ = "repuesto"` explicito a la clase Repuesto
- Restaurar la clase `OtRepuestoUtilizado` con `__tablename__ = "otrepuestoutilizado"`
- Verificar que los campos de Repuesto incluyen los 14 campos actuales

##### Paso 1.2: Corregir FK en models/documentos.py

- Cambiar `foreign_key="ordenestrabajo.id"` por `foreign_key="ordentrabajo.id"`
- Agregar `__tablename__ = "documento_adjunto"` explicito

##### Paso 1.3: Actualizar models/\_\_init\_\_.py

- Descomentar la importacion de `OtRepuestoUtilizado`
- Agregar `OtRepuestoUtilizado` y `Herramienta` a `__all__`

##### Paso 1.4: Crear models/herramientas.py

- Definir `class Herramienta` con `__tablename__ = "herramienta"`
- Campos: `nombre_herramienta`, `numero_identificacion`, `descripcion`, `categoria`, `cantidad_disponible`, `unidad_medida`, `ubicacion_almacen`, `estado_uso`, `costo_adquisicion`, `fecha_adquisicion`, `proveedor_ultimo`, `observaciones`, `imagen_ruta`, `fecha_registro`

#### Verificacion Fase 1:

`python -m uvicorn main:app --reload` debe iniciar sin errores. La tabla herramienta se crea vacia. Las OTs y documentos existentes funcionan normalmente.

## Fase 2: Actualizar schemas y rutas existentes

Esta fase actualiza los schemas Pydantic y las rutas de repuestos para que soporten los campos nuevos que ya existen en el modelo pero no estan expuestos en la API.

### Paso 2.1: Actualizar schemas/repuesto.py

- Agregar 7 campos faltantes a RepuestoCreate, RepuestoRead y RepuestoUpdate
- Campos: numero\_serie, especificaciones\_tecnicas, categoria, estado\_uso, proveedor\_ultimo, fecha\_ultima\_entrada, precio\_referencia

### Paso 2.2: Crear schemas/herramienta.py

- Definir HerramientaCreate, HerramientaRead, HerramientaUpdate
- Incluir todos los campos del modelo Herramienta

### Paso 2.3: Actualizar api/routes/repuestos.py

- Verificar que usa los schemas actualizados
- Agregar importacion de `utils.plantilla_inventario`
- Endpoint `/plantilla-excel` usa la plantilla combinada

### Verificacion Fase 2:

GET `/repuestos/` devuelve todos los campos. POST `/repuestos/` acepta campos nuevos.  
GET `/repuestos/plantilla-excel` descarga archivo combinado.

## Fase 3: Implementar modulo Herramientas completo

Esta fase crea toda la funcionalidad nueva del modulo Herramientas: rutas API, importacion Excel, plantillas, soporte en documentos, y configuracion.

### Paso 3.1: Crear api/routes/herramientas.py

- CRUD completo: POST `/`, GET `/`, GET `/id`, PUT `/id`, DELETE `/id`
- Plantilla CSV: GET `/plantilla-csv`
- Plantilla Excel combinada: GET `/plantilla-excel`
- Importar desde Excel/CSV: POST `/import-excel`
- Subir imagen: POST `/id/upload_imagen`
- Eliminar imagen: DELETE `/id/imagen`

### Paso 3.2: Crear utils/plantilla\_inventario.py

- Funcion `generar_plantilla_inventario_excel()` que genera un Excel con 3 hojas
- Hoja 1: Repuestos (encabezado azul oscuro #2C3E50)

- Hoja 2: Herramientas (encabezado verde oscuro #1B4332)
- Hoja 3: Instrucciones de uso para ambos tipos

#### Paso 3.3: Actualizar api/routes/documentos.py

- Agregar soporte para herramienta\_id en subir\_documento y listar\_documentos
- Estructura de carpetas: uploads/INVENTARIO/H0001\_Nombre/DOC/

#### Paso 3.4: Actualizar main.py

- Agregar `from api.routes import herramientas`
- Agregar `app.include_router(herramientas.router)`
- Agregar Herramienta a los imports de modelos

#### Paso 3.5: Actualizar database.py

- Agregar `_migrate_repuesto_columns()` para agregar columnas nuevas a la tabla existente
- Agregar `_migrate_documento_columns()` para agregar herramienta\_id

#### Paso 3.6: Actualizar config.py

- Agregar key `"herramientas_imagenes": "uploads/HERRAMIENTAS"` en directorios
- Borrar config.json existente para que se regenere con los defaults

#### Verificacion Fase 3:

GET /herramientas/ devuelve lista vacia. POST /herramientas/ crea herramienta. POST /documentos/ acepta herramienta\_id. El servidor arranca sin errores.

## Fase 4: Rediseñar frontend InventarioView

Esta fase transforma la pagina de Inventario de una vista solo-Repuestos a una vista con tabs internos (patron Reportes), agregando soporte completo para Herramientas.

#### Paso 4.1: Rediseñar layout con tabs internos

- Agregar `activeTab = ref("repuestos")` como estado
- Crear tabs-bar con dos tabs: "Repuestos" y "Herramientas"
- Usar `v-if` para alternar contenido segun el tab activo
- Patron visual: boton activo con fondo azul claro y borde inferior azul

#### Paso 4.2: Mover controles al lado derecho

- Top-bar: titulo del tab a la izquierda, search + Cargar + Agregar a la derecha
- Patron consistente con las paginas de Equipos y Ordenes

### Paso 4.3: Implementar CRUD de Herramientas

- Duplicar logica de repuestos adaptada para herramientas
- Formulario de creacion/edicion con campos de Herramienta
- Modal de detalle con la informacion completa
- Modal de importacion Excel/CSV

### Paso 4.4: Agregar documentacion a Herramientas

- Integrar componente DocumentosAdjuntos en el modal de detalle
- Pasar :herramientaId como prop

### Paso 4.5: Agregar "Otro" a los selects

- Categoria repuesto: Filtro, Lampara, Electrodo, Reactivo, Sensor, Cable, Bateria, Otro
- Estado de uso: Disponible, En Uso, En Reparacion, Dado de Baja, Otro
- Unidad: unidad, par, metro, litro, kit, paquete, rollo, Otro
- Categoria herramienta: Instrumento de Medicion, Herramienta Manual, Consumible, Kit, Otro

### Verificacion Fase 4:

Los tabs funcionan correctamente. Se puede crear, editar, eliminar, importar y ver documentos tanto de repuestos como de herramientas. Los selectores incluyen la opcion "Otro".

## Fase 5: Pruebas finales y limpieza

La fase final verifica que todo el sistema funciona de punta a punta y realiza limpieza de codigo y configuracion.

### Paso 5.1: Pruebas de integracion manual

- Crear un repuesto con todos los campos
- Crear una herramienta con todos los campos
- Subir imagen a un repuesto y a una herramienta
- Subir documento a un repuesto y a una herramienta
- Descargar plantilla Excel y CSV
- Importar datos desde Excel
- Editar y eliminar registros
- Verificar que las OTs existentes siguen funcionando

### Paso 5.2: Limpieza de \_\_pycache\_\_

`rmdir /s /q backend\models\__pycache__` y lo mismo para routes, schemas, utils. Esto garantiza que Python cargue los archivos modificados y no las versiones cacheadas.

### Paso 5.3: Regenerar config.json

Borrar `backend/config.json` y reiniciar el servidor. El archivo se regenera automaticamente con los defaults actualizados, incluyendo `herramientas_imagenes`.

### Paso 5.4: Commit a GitHub

- Hacer commit con mensaje descriptivo por cada fase completada
- Verificar que no se incluyen archivos temporales ni `__pycache__`

## 4. Referencia: Nombres de Tabla y Foreign Keys

Es critical que cada modelo defina `__tablename__` explicitamente y que todas las foreign keys referencien el nombre correcto. A continuacion se muestra la tabla de referencia con los nombres correctos para cada modelo.

```
Usuario
Equipo: estado_id, estado_id
EstadoEquipo
EstadoOT
Equipo: estado_id, usuario_id, tareapreventiva
Repuesto
OrdenRepuesto: repuesto_id
Equipo: responsable_id
TareaPreventiva: repuesto_id
Evento: trabajo_id, trabajo_id, tecnico_id
OrdenRepuesto: repuesto_id, repuesto_id, herramienta_id
Herramienta
```

## 5. Estrategia de Migracion de Base de Datos

Como el proyecto usa SQLite sin herramientas de migracion (Alembic, etc.), las migraciones se manejan manualmente con ALTER TABLE. El patron utilizado es verificar si cada columna existe antes de agregarla, lo que permite que la aplicacion funcione tanto con bases de datos nuevas como con existentes.

### 5.1 Migraciones para la tabla repuesto

| Columna                   | Tipo SQL | Accion                                |
|---------------------------|----------|---------------------------------------|
| numero_serie              | VARCHAR  | ALTER TABLE repuesto ADD COLUMN       |
| especificaciones_tecnicas | VARCHAR  | ALTER TABLE repuesto ADD COLUMN       |
| proveedor_ultimo          | VARCHAR  | ALTER TABLE repuesto ADD COLUMN       |
| fecha_ultima_entrada      | VARCHAR  | ALTER TABLE repuesto ADD COLUMN       |
| precio_referencia         | REAL     | ALTER TABLE repuesto ADD COLUMN       |
| categoria                 | VARCHAR  | ALTER TABLE repuesto ADD COLUMN       |
| estado_uso                | VARCHAR  | ALTER TABLE repuesto ADD COLUMN       |
| codigo_equivalente        | -        | DROP COLUMN (si existe, SQLite 3.35+) |

## 5.2 Migraciones para la tabla documento\_adjunto

| Columna        | Tipo SQL                           | Accion                                   |
|----------------|------------------------------------|------------------------------------------|
| herramienta_id | INTEGER REFERENCES herramienta(id) | ALTER TABLE documento_adjunto ADD COLUMN |

## 5.3 Tablas nuevas (creadas automaticamente)

| Tabla               | Modelo              | Creada por                     |
|---------------------|---------------------|--------------------------------|
| herramienta         | Herramienta         | SQLModel.metadata.create_all() |
| otrepuestoutilizado | OtRepuestoUtilizado | SQLModel.metadata.create_all() |

Importante: Las migraciones se ejecutan automaticamente al iniciar el servidor, dentro de la funcion `create_db_and_tables()` en `database.py`. Cada migracion verifica si la columna ya existe antes de agregarla, por lo que es segura ejecutarla multiples veces.

# 6. Notas Importantes

## 6.1 Orden estricto de ejecucion

Las fases DEBEN ejecutarse en orden. No se puede implementar el frontend (Fase 4) sin que el backend funcione (Fase 3). No se puede crear la ruta de herramientas (Fase 3) sin que los modelos y schemas esten correctos (Fase 2). Y no se puede hacer nada si el servidor no arranca (Fase 1).

## 6.2 Hacer backup de la base de datos

Antes de iniciar cualquier modificacion, hacer una copia del archivo `cmms_bioai.db`. Aunque las migraciones estan disenadas para ser seguras, siempre es mejor tener un respaldo.

## 6.3 Limpiar `__pycache__` antes de cada prueba

Despues de modificar cualquier archivo Python, eliminar los directorios `__pycache__` para garantizar que Python cargue los archivos modificados:

```
rmdir /s /q backend\models\__pycache__
```

```
rmdir /s /q backend\api\routes\__pycache__
```

```
rmdir /s /q backend\schemas\__pycache__
```

```
rmdir /s /q backend\utils\__pycache__
```

## 6.4 No eliminar la base de datos

No borrar `cmms_bioai.db` para "empezar de nuevo". Las migraciones estan disenadas para agregar columnas a la base de datos existente. Si se borra la base de datos, se pierden todos los datos de equipos, OTs y repuestos que ya existen. Solo como ultimo recurso si la base de datos esta corrupta.

## 6.5 Regenerar `config.json`

Despues de modificar `config.py`, borrar `config.json` para que se regenere con los defaults actualizados. Si no se borra, el archivo existente no tendra la nueva key `herramientas_imagenes`.